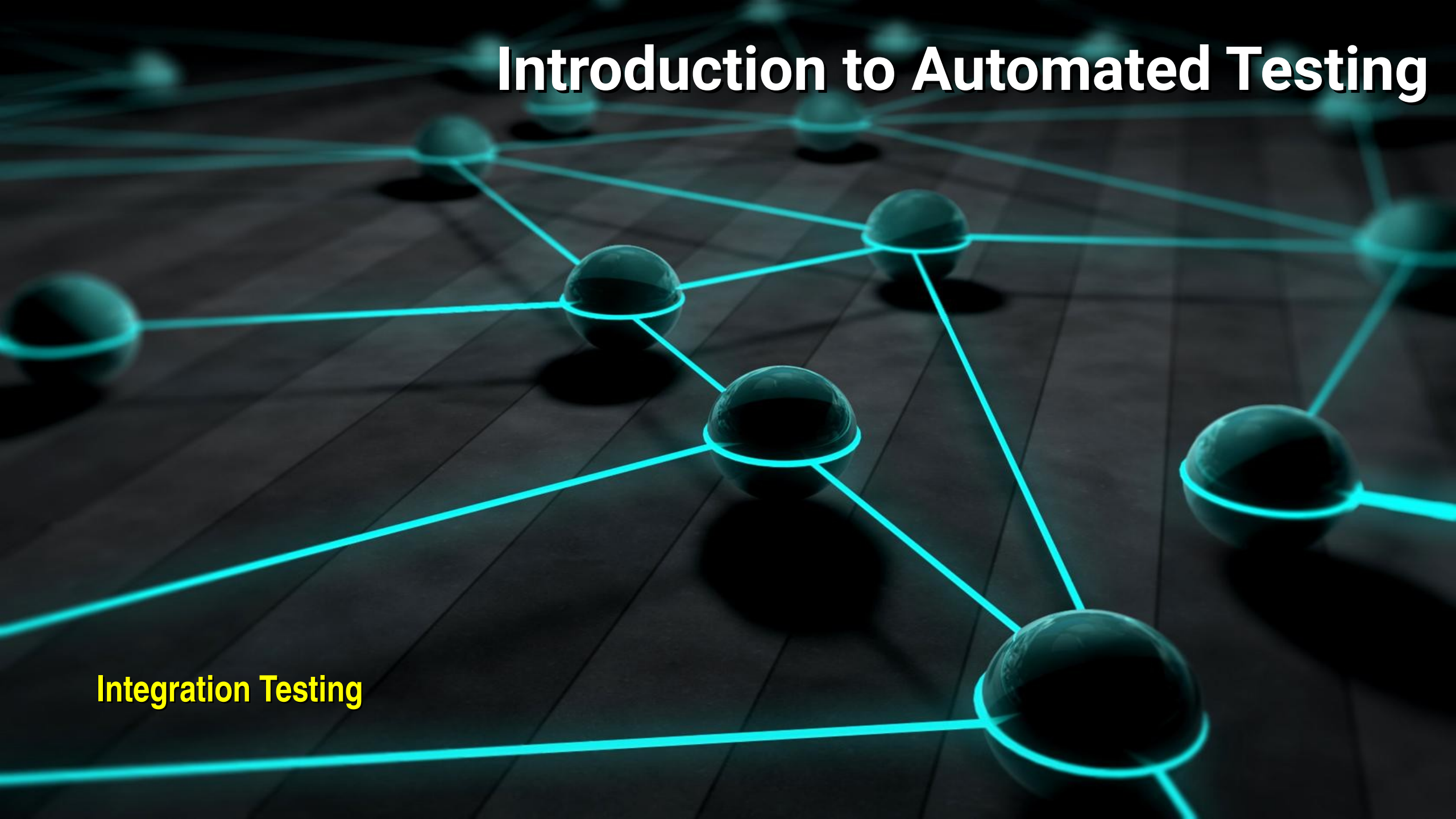


Introduction to Automated Testing

A network diagram consisting of several glowing blue spheres connected by thin, glowing blue lines. The spheres are arranged in a non-uniform pattern across the frame, with some in the foreground and others receding into the background. The lines connect the spheres, creating a web-like structure. The entire scene is set against a dark, textured background that resembles a wooden floor with vertical planks.

Integration Testing

Integration Testing

- Integration testing verifies the interactions between components or systems.
 - It does not test what a component does internally.
 - That is done by unit testing
 - It tests what happens at the seams
 - The connections between unit tested components.
- The question it answers:
 - "These pieces work individually. Do they work together?"
- Also called build testing



Why Unit Tests Are Not Enough

- Unit tests
 - Verify a component against your belief about its dependencies
 - Mocks encode assumptions. Integration tests test assumptions.
 - Example on the right, the unit tests pass forever even if
 - The real query is malformed SQL
 - The column was renamed
 - The real method throws on a null ID
 - The transaction never commits
- Integration tests
 - Test the dependencies between components
 - And how errors may be propagated

Where It Sits

Level	Scope	Speed	What it catches
Unit	One class or function, isolated	ms	Logic errors
Integration	Two or more components together	10ms - 10s	Interface and contract errors
E2E	The whole system, user's view	10s - minutes	Workflow errors

```
// Unit test: mocks the repository  
when(repo.findById(42L)).thenReturn(Optional.of(customer));
```



Two Levels (ISTQB)

- Component integration testing
 - Between modules inside one system
 - Your service layer + your repository + your database
 - Usually developer-owned, runs with the build
- System integration testing
 - Between separate systems
 - Your service + the payment gateway + the CRM
 - Often crosses team or organizational boundaries



Strategies and Test Doubles

- Big bang integration
 - Build everything
 - Connect everything
 - Test everything at once.
 - [A] [B] [C] [D] [E] → assemble all → test → 
 - Why it is ineffective
 - A failure could originate anywhere
 - Debugging is archaeology – which component(s) failed
 - Nothing testable until everything is finished
 - Defects found latest and at a higher cost
 - Almost never the right choice
 - But it is what happens by default.
 - Mostly because it takes minimal planning and effort



Strategies and Test Doubles

- Incremental integration
 - Add one component at a time
 - Test after each addition.
 - Top-down
 - Start at the top, use stubs for what is below
 - Bottom-up
 - Start at the bottom, use drivers to call upward
 - Hybrid
 - Both directions, meet in the middle
 - When a test fails after adding component X
 - The defect is almost certainly in X or its interface.

Stubs and Drivers

TOP-DOWN

```
[ Real UI Layer ]  
    |  
[ Real Service ]  
    |  
[ STUB Database ]  
  (canned answers)
```

BOTTOM-UP

```
[ Driver - fake caller ]  
    |  
[ Real Service ]  
    |  
[ Real Database ]
```

- **Stub** — stands in for something your code *calls*
- **Driver** — stands in for something that *calls* your code



Strategies and Test Doubles

- Narrow vs broad integration tests
 - Narrow
 - Exercises one small slice of the code that talks to a dependency
 - Uses a test double for that dependency
 - Fast, deterministic, runs anywhere
 - Broad
 - Exercises real running instances of multiple components
 - Real database, real HTTP calls
 - Slower, more realistic, more infrastructure
 - Both are integration tests
 - They live at different pipeline stages.



Strategies and Test Doubles

- The test double taxonomy
 - Same as what we saw in unit testing
 - We use doubles to support integration testing
 - Drivers and stubs are doubles that act as caller or callee
 - These can be any of the double types we already saw
 - As our testing proceeds, we replace doubles with fully unit tested components

Double	Behaviour
Dummy	Passed but never used — fills a parameter slot
Stub	Returns canned answers
Spy	A stub that also records how it was called
Mock	Pre-programmed with expectations; fails if not met
Fake	Real working implementation, unsuitable for production



Strategies and Test Doubles

- Real dependencies: Testcontainers
 - The industry standard for broad integration tests
 - Spin up a real PostgreSQL, Kafka, or Redis in a container, per test run
 - Real engine, real SQL dialect, real behaviour
 - Disposable and isolated
 - Eliminates the fidelity gap

```
@Container
static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16");
```



Strategies and Test Doubles

- Stubbing HTTP Dependencies
 - For third-party APIs you cannot or should not call:
 - WireMock
 - Runs an in-process HTTP server returning scripted responses
 - Also lets you simulate what the real service will not do on demand:
 - Timeouts and slow responses
 - 500s and 503s
 - Malformed payloads
 - Rate limiting
 - Testing failure paths matters as much as testing success paths.

```
stubFor(get("/api/rates")  
        .willReturn(okJson("{\"USD\": 1.34}"));
```



Integration Testing in CI/CD

- Integration tests appear in both stages
- The pipeline stages

COMMIT STAGE
< 10 minutes

Compile
Unit tests
NARROW integration
Static analysis
Quality gate

ACCEPTANCE STAGE
30-60 minutes

Broad integration
Contract tests
Key E2E journeys
Performance

DEPLOY
on demand

Smoke tests
Canary
Monitoring

Test	Stage	Why
Service + in-memory DB	Commit	Fast, no infrastructure
Service + WireMock	Commit	Fast, deterministic
Service + real PostgreSQL	Acceptance	Needs a container
Two services over HTTP	Acceptance	Needs deployment
Full checkout journey	Acceptance	Slow, broad



Integration Testing in CI/CD

- Environments and data in the pipeline
 - Environment
 - Ephemeral per run is ideal
 - Shared environments create cross-run interference
 - Never point CI at a shared "QA" database and expect determinism
 - Data
 - Seed known fixtures before each run
 - Isolate: per-test transaction rollback, or per-run schema
 - Clean up in teardown and assume teardown may not run



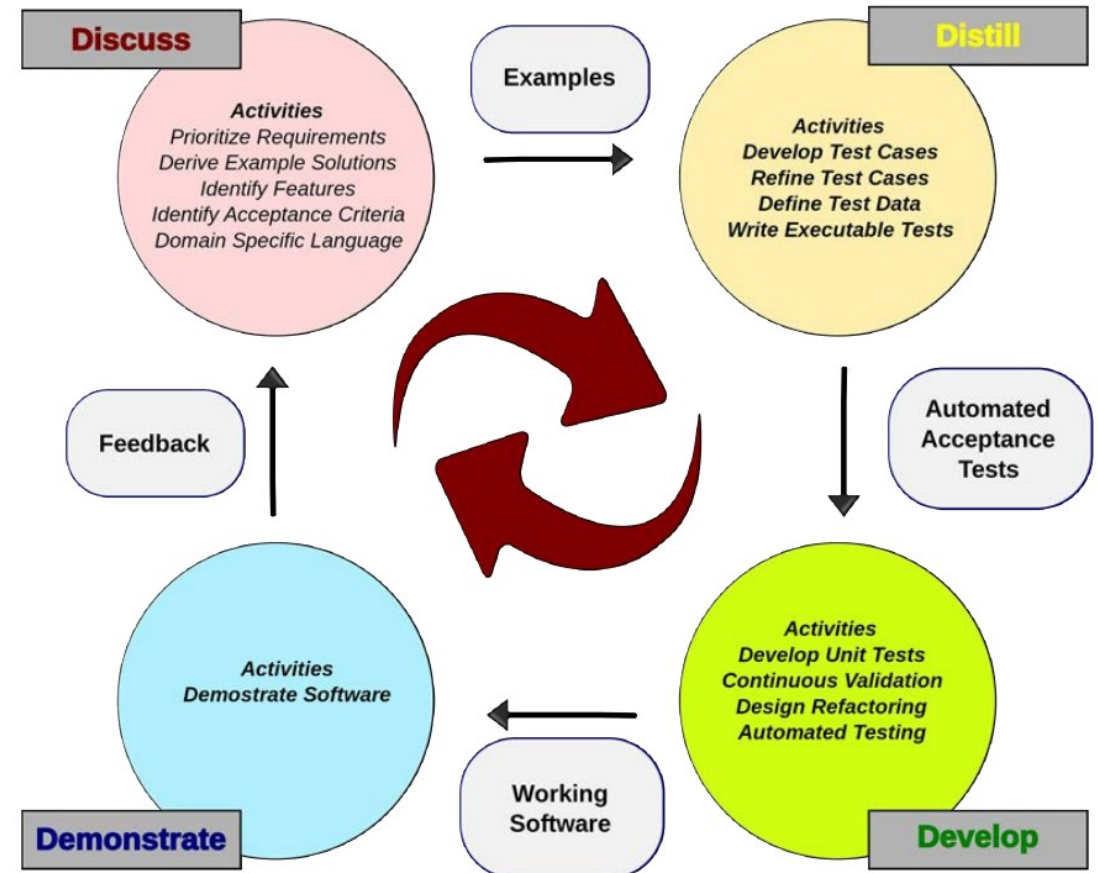
Integration Testing in CI/CD

- When to test
 - Unit tests are run after every code change
- Integration tests commit phase
 - Can be run by a developer before submitting pull request
 - Can be run by the integration team in order to validate merged code
 - Often run on a specific schedule
 - For example, every morning at 2am
- Integration tests acceptance phase
 - Usually scheduled at less frequent intervals due to resource usage
 - For example, weekly build and integration test.



BDD and Cucumber

- BDD and ATDD are not testing tools
 - Behaviour Driven Development
 - Acceptance Test Driven Development
 - Collaboration practices for three groups
 - Business: what problem are we solving?
 - Development: how will we build it?
 - Testing: how could this go wrong?
 - They agree on examples of desired behaviour, in shared language, before the code is written.
 - Cucumber automates those examples
 - It does not create them.



BDD and Cucumber

- Gherkin structure
 - A way of encoded acceptance tests
 - Uses natural language
 - Allows all groups to agree on what an acceptance test should be and what the correct behaviour is
 - The tests are then automated
 - Cucumber then executes the automated tests

Feature: Account withdrawal

As a customer

I want to withdraw cash

So that I can spend it

Background:

Given I have a current account

Scenario: Successful withdrawal within balance

Given my account balance is £500

When I withdraw £100

Then my balance should be £400

And I should receive £100

Scenario Outline: Withdrawal limits

Given my account balance is <balance>

When I withdraw <amount>

Then the withdrawal should be <result>

Examples:

balance	amount	result
500	100	approved
500	500	approved
500	501	declined
0	1	declined



BDD and Cucumber

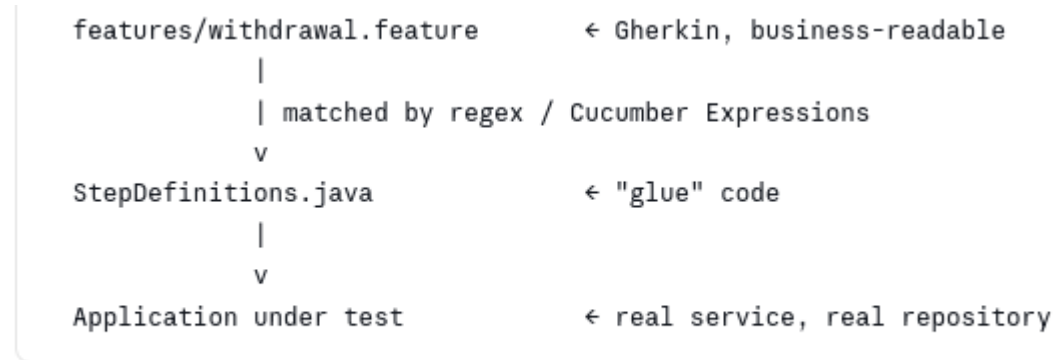
- Gherkin Keywords

Keyword	Purpose
Feature	The capability being described
Scenario	One concrete example
Given	Precondition — put the system in a known state
When	The action under test
Then	The expected outcome
And / But	Continue the previous keyword
Background	Shared Given steps for every scenario
Scenario Outline + Examples	Same scenario, many data sets
@tag	Filter and group scenarios



BDD and Cucumber

- Cucumber-JVM Architecture



The **runner** ties them together and reports results.



BDD and Cucumber

- Step definitions in Java
 - For each Gherkin statement
 - The pattern of the statement is matched to a piece of executable code called a step definition
 - These step definitions are parameterized so that the same code can be used in different steps
 - The Cucumber tool creates the stubs for the step definitions when it compiles the Gherkin
 - When the tests are executed
 - Each Gherkin statement is read
 - The corresponding step definition is executed
 - The code uses assertions in the same way as Junit to determine if a test passes or fails

```
public class AccountSteps {  
  
    private Account account;  
  
    @Given("my account balance is £{int}")  
    public void myAccountBalanceIs(int balance) {  
        account = new Account(balance);  
    }  
  
    @When("I withdraw £{int}")  
    public void iWithdraw(int amount) {  
        account.withdraw(amount);  
    }  
  
    @Then("my balance should be £{int}")  
    public void myBalanceShouldBe(int expected) {  
        assertEquals(expected, account.getBalance());  
    }  
}
```



BDD and Cucumber

- Declarative vs imperative Gherkin
 - Declarative form is interface independent
 - Allows the same Gherkin files to be used with different implementations
 - The step definitions are the imperative implementations of the declarative Gherkin
 - The same tests can be used for different implementations like web, mobile, etc
 - The difference is in the implementation step definitions
- Gherkin is “functional”
 - It describes what happens
 - Not how it happens

gherkin

```
# IMPERATIVE - a script in Gherkin costume
Given I navigate to "/login"
When I type "user@test.com" into field "#username"
And I type "hunter2" into field "#password"
And I click the button with class ".btn-submit"
Then element "#welcome" should contain "Hello"
```

gherkin

```
# DECLARATIVE - business intent
Given I am a registered customer
When I sign in with valid credentials
Then I should see my account dashboard
```



BDD and Cucumber

- Cucumber cost/benefit
 - Cucumber adds a translation layer
 - Gherkin matches with regex maps to Java.
 - The layer costs maintenance
 - It is worth paying only if non-technical people actually read the scenarios.
 - BDD without the conversation is just an expensive test syntax.
 - If your Gherkin is written by testers, read by nobody, and executed by CI, then just use plain Junit
 - It is faster to write and simpler to maintain.

gherkin

```
# IMPERATIVE – a script in Gherkin costume
Given I navigate to "/login"
When I type "user@test.com" into field "#username"
And I type "hunter2" into field "#password"
And I click the button with class ".btn-submit"
Then element "#welcome" should contain "Hello"
```

gherkin

```
# DECLARATIVE – business intent
Given I am a registered customer
When I sign in with valid credentials
Then I should see my account dashboard
```



Best Practices

- Test at the right level
 - Before writing an integration test, ask:
 - "Could a unit test verify this?"
 - Business logic and calculations should be tested at the unit level
 - Validation rules should be tested at the unit level
 - Verifying inter-module function calls, like SQL, actually work are integration tests
 - Verifying the API contracts hold are integration tests
 - Verifying work flow scenarios are E2E tests
- Integration tests that verify business logic are slow unit tests.



Best Practices

- Make tests independent
 - Every test must pass alone, and in any order
 - The technical term is “combinatorial tests”
- Techniques:
 - Roll back a transaction after each test
 - Truncate and reseed between tests
 - Generate unique data per test
 - (“ann-” + UUID)

```
// WRONG – Test B depends on Test A's leftovers
@Test void a_createsCustomer() { repo.save(new Customer("Ann")); }
@Test void b_findsCustomer()  { assertNotNull(repo.find("Ann")); }
```



Best Practices

- Make failures diagnosable
 - An integration test failure should not require a manual reproduction.
 - Capture on failure:
 - The full request and response bodies
 - Application logs for the test window
 - Database state at failure
 - Correlation IDs across services
 - Scenario name and step that failed



Best Practices

- Test the unhappy paths
 - Most integration bugs live in error handling, which is the least-exercised code in the system.
 - For example
 - Dependency returns 500 server error
 - Dependency times out
 - Dependency returns malformed JSON
 - Dependency is unreachable
 - Database constraint violation
 - Concurrent update conflict
 - Stubbing tools exist largely to make these testable.



Best Practices

- Control everything non-deterministic
 - Integration tests run in the real world
 - The real world does not remain stable
 - It is a reliability failure to assume it is stable
 - Replace any variable components of the environment with fakes
 - Ensure the fakes are deterministic
 - In other words, we can replicate environmental conditions to be the same for each test run

Source	Control
System clock	Inject a fixed <code>Clock</code>
Random values	Seed the generator
Network calls	WireMock or a stub
Async completion	Poll with a timeout, never <code>sleep()</code>
Auto-increment IDs	Do not assert on them
Map / set iteration order	Do not assert on it



Common Mistakes

- Mocking everything
 - If every collaborator is a double, it is a unit test with extra steps.
 - An integration test must integrate something real.

```
@Test
void integrationTest() {
    when(repo.save(any())).thenReturn(customer);
    when(email.send(any())).thenReturn(true);
    when(audit.log(any())).thenReturn(true);
    // ... what exactly is being integrated?
}
```



Common Mistakes

- Testing logic at the wrong level
 - Symptom: 200 Cucumber scenarios covering validation rules.
 - Each takes 4 seconds. A unit test takes 4 milliseconds.
 - That is a 1000× cost for the same coverage.

Scenario: Postcode with lowercase letters is accepted

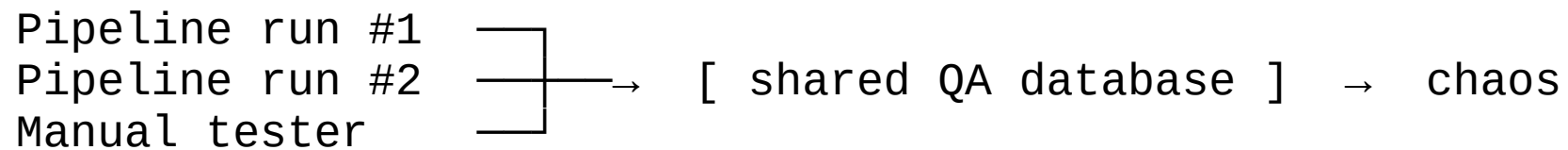
Scenario: Postcode with no space is accepted

Scenario: Postcode with two spaces is rejected



Common Mistakes

- Shared mutable environment
 - Symptoms: tests pass when run alone but fail in CI
 - Failures that never reproduce, "just re-run it."
- Isolation is the precondition for determinism.



Common Mistakes

- Big bang integration
 - Waiting until everything is built before testing any of it.
 - Defects found at the latest, most expensive moment
 - Root cause could be anywhere
 - Schedule pressure at exactly the wrong time
- Integrate continuously, in small increments.



Common Mistakes

- No cleanup, or cleanup that never runs
- Defensive strategies:
 - Clean before the test as well as after
 - Use transaction rollback rather than deletion
 - Unique data per run, so leftovers cannot collide
 - Ephemeral environments, so leftovers cannot survive



Common Mistakes

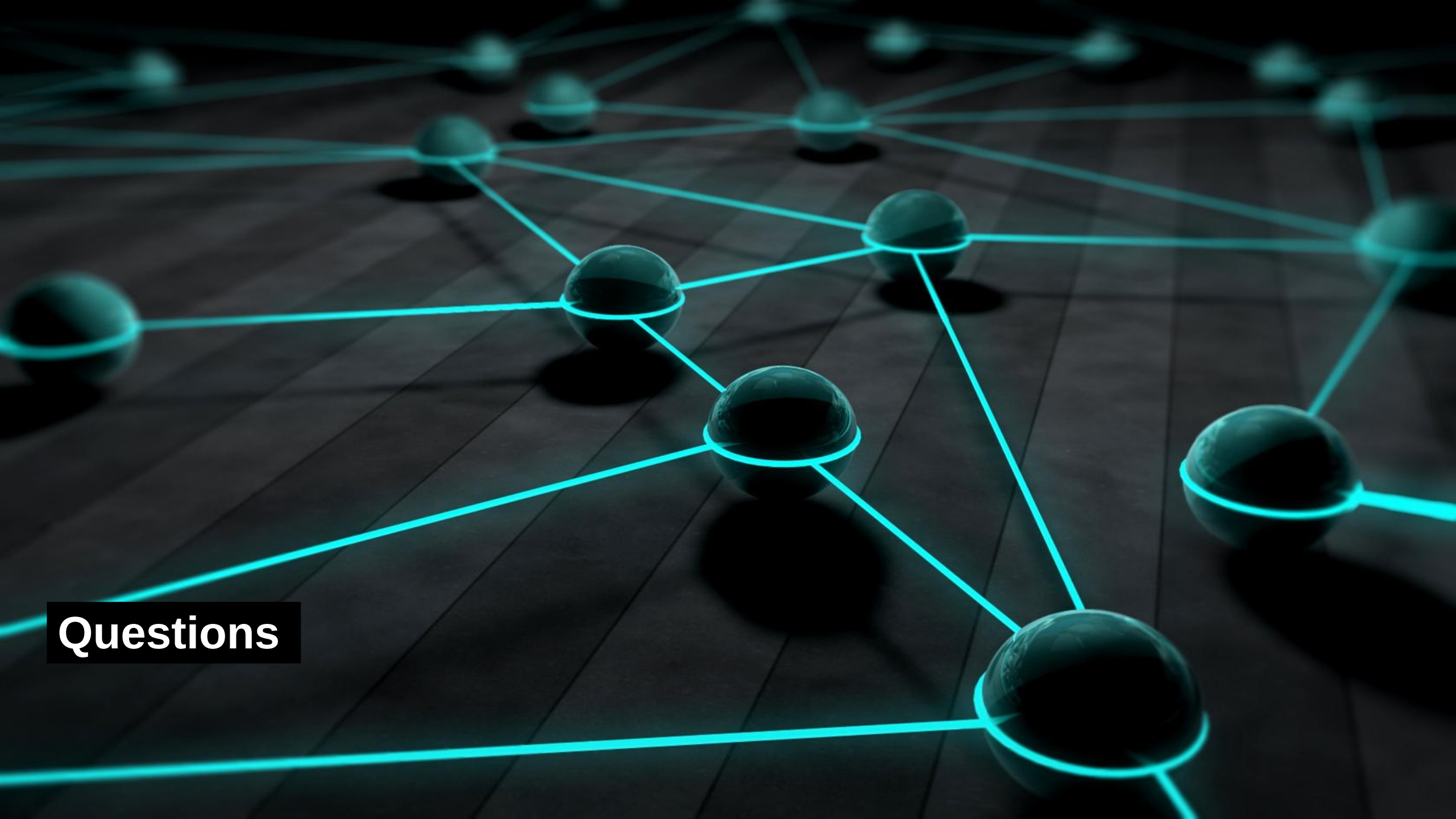
- Cucumber Anti-Patterns
 - Imperative Gherkin: UI clicks written as business language
 - Conjunction steps: Given I am logged in and have 3 items and...
 - Scenario coupling: Scenario 2 depends on Scenario 1
 - Technical Gherkin: Given the SQL table users contains 3 rows
 - Feature files nobody reads: the whole effort, with none of the value
 - Step explosion: 400 near-duplicate step definitions
 - Over-abstracted steps: Given the system is set up (set up how?)



Common Mistakes

- Slow tests in the commit stage
 - A 40-minute commit stage means:
 - Developers stop waiting for it
 - They batch changes to avoid it
 - Batches make failures harder to localize
 - Which makes people trust it less
 - A pipeline stage nobody waits for provides no feedback.
 - Move broad integration tests to the acceptance stage.





Questions